

System Architecture Overview

The proposed system employs a **modular, microservice-based architecture** to enable independent development and testing of each detection and educational component. Input videos are handled by a video-processing service that decodes clips, extracts frames (using OpenCV/FFmpeg), normalizes resolutions, and performs face detection or preprocessing. Audio tracks are handled by a separate audio-processing service that extracts raw audio, computes spectrograms or MFCC features, and normalizes them for analysis. These features are fed into dedicated neural-network services: a **CNN feature extractor** (e.g. Xception or ResNet-based models) for spatial frame analysis, and a **temporal sequence model** (e.g. LSTM/GRU or transformer/ViT) for modeling sequential dependencies across frames. A **fusion service** then combines audio and visual embeddings – for example by late concatenation or attention-based fusion – and passes them to a classifier. An **explainability module** taps into the CNN and temporal models to generate Grad-CAM heatmaps on video frames and per-frame saliency logs. An additional **metadata analyzer** service parses EXIF tags and any available social-media metadata (timestamps, camera model, geolocation) to flag inconsistencies or anomalies (for example, metadata indicating post-dating content capture). Finally, a **FastAPI backend** exposes these services via RESTful APIs, while a **Next.js single-page frontend** provides the user interface (upload forms, result dashboards, and the sandbox). User accounts, subscription status, and logs of all analyses are stored in a PostgreSQL database. This high-level component diagram (Figure 1) illustrates the modular pipeline:

Figure 1: Example deepfake-detection pipeline (from Ismail et al. 2023). Video and audio inputs are preprocessed separately: video frames are fed to a CNN (e.g. Xception) and audio to a spectrogram-CNN (e.g. InceptionResNet). The feature embeddings are concatenated and passed through a GRU and MLP for final classification.

This modular design – with each service containerized in Docker – allows the video, audio, CNN, temporal, fusion, explainability, and metadata components to be developed and scaled independently. FastAPI (Python) is chosen for the backend due to its high performance and native support for ASGI and asynchronous endpoints, making it ideal for ML inference services. The frontend uses Next.js (React) for a fast, SEO-friendly SPA with full routing; user authentication can be handled via NextAuth or OAuth2 tokens. The database (PostgreSQL) tracks user profiles, subscription levels, and detailed logs of inputs and detection outcomes. All services run in Docker containers (using NVIDIA's CUDA base images for GPU support) and can be orchestrated via Docker Compose for development and Kubernetes/ECS in production. CI/CD pipelines (e.g. GitHub Actions) automate testing (pytest suite) and deployment of each service image to AWS (EC2 or ECS/EKS) on merge, ensuring continuous integration and delivery.

Core Component Descriptions

- **Video/Frame Processing Module:** This service ingests uploaded video files and performs decoding and frame extraction (e.g. using OpenCV or FFmpeg). It converts videos to a consistent frame rate/resolution, optionally applies face detection or cropping, and outputs raw image sequences. This module also handles low-level augmentation (normalization, compression artifact detection, resizing) to prepare inputs for the CNN feature extractor. Because it is stateless and I/O-bound, it can be scaled horizontally (e.g. multiple containers pulling from an S3 queue).
- **Audio Processing Module:** A separate service isolates audio from the video (or accepts audio-only input). It preprocesses the audio by resampling, denoising, and computing features such as spectrograms, MFCCs, or constant-Q transforms. These features are normalized and possibly segmented (e.g. into frames of 1–2 seconds) before being fed to neural models. The pipeline follows the standard audio-detection architecture: a front-end feature extractor and a back-end classifier. This modular approach (pipeline with feature extractor + classifier) is well-established as a “de facto” standard for audio deepfake detection.
- **CNN Feature Extractor:** For visual frames, a convolutional neural network (e.g. Xception, ResNet variants, or EfficientNet) encodes spatial artifacts of manipulation. This service can support multiple model backends – for instance, an XceptionNet trained on face-swapped images and an EfficientNet trained on inpainting artifacts – and output rich feature vectors. The network’s outputs for each frame (or per-frame patches) are passed on to the temporal model or combined immediately by the fusion logic. The chosen models (e.g. Xception) should be preloaded into GPU-backed containers for fast inference. During development, these models are trained off-line on cloud GPU instances (P3/G4 on AWS) and versioned (e.g. via model registry or as part of Docker images) for production use.
- **Temporal Sequence Model:** To capture temporal inconsistencies across frames, a sequential model is used. Options include recurrent networks (LSTM/GRU) or transformer-based architectures (e.g. Vision Transformer variants) that process sequences of frame embeddings. For example, a GRU network can take the concatenated embeddings of several consecutive frames and produce a context-aware representation. Alternatively, a ViT applied to a sequence of frame patch embeddings could model long-range dependencies. The temporal model’s output (sequence-level encoding) is forwarded to the fusion or classifier. Using a transformer may improve generalization to varied motion patterns, whereas LSTM/GRU are simpler and well-understood. Regardless, the module is isolated: it receives feature vectors (and optionally

audio embeddings) and returns a temporal representation or classification confidence.

- **Audio-Visual Fusion Logic:** This component combines the outputs of visual and audio sub-models. A simple approach is *late fusion*: concatenate the final confidence scores or feature vectors from audio and video branches and feed them to a dense network for final real/fake classification. A more sophisticated approach uses attention: for example, an attention layer can weigh audio and video features differently per example, or a multimodal transformer can jointly attend to both modalities. The fusion service might also implement logic like weighted voting or threshold logic if one modality is missing (e.g. silent videos). The fusion model is itself trained on multimodal datasets. Careful calibration (e.g. via a validation set) is needed so that neither modality overwhelmingly dominates. The fused output is a probability or logit indicating the likelihood of the input being a deepfake.
- **Explainability Module:** To provide transparency, this service computes saliency maps and textual explanations. For visual models, Grad-CAM (or GradCAM++) is applied to the CNN feature extractor's final convolutional layers to highlight image regions influencing the prediction. These heatmaps (per-frame overlays) are stored or returned to the user interface, allowing humans to see why the model flagged certain frames. Simultaneously, per-frame detection scores and metadata can be logged; for example, flagging frames with unusually high confidence or odd audio/video synchronization. A text-based saliency log might describe features ("blurry edges around mouth" or "inconsistent shadows") using either heuristic rules or a small language model trained on detection annotations. This explainability pipeline is optional but critical for trust – Grad-CAM has been shown to be a generic method to back-propagate gradients and obtain saliency maps in vision tasks. All explainability outputs are archived and tied to the analysis report for auditability.
- **Metadata Analyzer:** This microservice extracts embedded EXIF data from images (e.g. camera model, capture timestamp, GPS location) and harvests social-media metadata (e.g. upload date, author tags) if a URL is provided. It then analyzes inconsistencies: for instance, if EXIF timestamps predate the claimed creation date or if compression signatures don't match the declared device. As Amped's forensic analysis notes, metadata (EXIF, container format details, file hashes) often reveal tampering or provenance issues. The service may cross-check face-recognition data (if configured) against known identity databases or verify whether the face appears in known deepfake corpora. Any anomalies are appended to the report. The metadata analyzer runs independently and can flag

issues even if the neural detector is uncertain, providing a crucial “traditional forensics” layer.

- **Web Interface (Next.js SPA):** The frontend is a multi-page Next.js application. Key pages include: an upload page for submitting videos, a dashboard showing analysis results (with frame-by-frame video playback, Grad-CAM overlays, and audio waveform), a sandbox page for the educational deepfake demo, and user account pages (profile, subscription, usage history). Next.js provides server-side rendering (SSR) for fast initial load and SEO, and client-side navigation for a smooth user experience. API routes on Next.js can handle authentication (NextAuth with JWT or OAuth2) and proxy requests to the FastAPI backend. Static assets (like model files for the sandbox) are managed via Next’s static file support or a CDN. The UI fetches results from FastAPI’s REST endpoints and displays them with charts and video players. Responsive design and accessibility are enforced for usability.
- **User Accounts, Subscriptions, and Logs (PostgreSQL):** A centralized Postgres database stores user profiles, hashed credentials, and subscription status (basic vs. premium features). Each analysis request is logged with user ID, timestamp, input metadata (file name, size), and results (classification score, explanation summary). This enables auditing, usage quotas, and billing. The schema might include tables like Users, Analyses, Subscriptions, and Logs. FastAPI services use an ORM (e.g. SQLAlchemy or Pydantic models) to interact with Postgres. For high availability, the database runs on a managed cloud instance or a Docker container in a dedicated network. Role-based access control ensures only authenticated users can invoke the detection API, and queries are parameterized to prevent SQL injection.
- **Educational Sandbox Environment:** To educate users, the platform includes a sandbox where users can *create* simple deepfakes in a controlled way. For example, the sandbox might let a user upload a selfie and a source audio clip, then run a pre-trained face-swap or lip-sync model in-browser (via TensorFlow.js/WebGL or a remote GPU container) to generate an output. This process is entirely contained: the sandbox can run in an isolated Docker container (or client-side sandbox) with strict limits on compute and without access to production models or sensitive APIs. All sandbox activity is logged and CAPTCHA/age-verification can be used to prevent abuse. CivAI’s Deepfake Sandbox project demonstrates that a safe, interactive demo can be offered publicly. Our sandbox similarly provides immediate feedback but with disclaimers; it educates users about how a deepfake is made and then feeds the result back into the detection pipeline so users “learn by doing.” The sandbox UI

is integrated into Next.js, but it may use separate backend routes and containers to ensure it cannot accidentally expose the main model code or user data.

Implementation Stack and Deployment

Backend Framework: All core services (video, audio, models, explainability, metadata) are implemented in Python using FastAPI. This framework natively supports asynchronous request handling and automatic OpenAPI docs. FastAPI is well-suited for ML inference: as one author notes, it easily integrates with ML libraries, offers built-in validation, CORS, and generates Swagger/Udemy docs for each endpoint. Each service can be a separate FastAPI app (or router) within its container.

Machine Learning (PyTorch, OpenCV): The CNN and temporal models are built with PyTorch. Training is performed offline on GPU instances; models are exported (state_dict) and loaded at inference time. During inference, frames are handled by OpenCV (in the video-processing service) and converted to tensors. Audio processing uses librosa or similar. GPU compatibility is ensured by using NVIDIA CUDA base images and the nvidia-docker runtime in containers. For example, AWS Deep Learning Container images (PyTorch on CUDA) can be used to simplify setup. Each ML service checks environment variables (via a config file) to select CPU vs GPU (for local dev vs AWS). Logging and error handling (via Python's logging or Sentry) ensure that failures in model loading or inference are captured.

Containerization (Docker): From day one, each component is containerized. A common Docker network and Docker Compose file allow local development of the multi-service app (video, audio, fusion, explainability, metadata, frontend, database). Each service has its own Dockerfile specifying the base image (e.g. nvidia/cuda:11.7-cudnn8-devel-ubuntu22.04 with Python 3.11 and PyTorch) and dependencies. A root docker-compose.yml orchestrates containers. This aligns with the microservices best practice: “each service in its own folder or repo... promoting isolation and independent deployment”. In production, containers are pushed to a registry (e.g. AWS ECR) and deployed to a Kubernetes cluster or ECS task set on AWS. Persistent state (Postgres) is either managed by AWS RDS or a container with attached volume.

AWS GPU Instances: For heavy inference and model training, GPU-backed EC2 instances (e.g. p3.2xlarge or g4dn.xlarge with NVIDIA T4 GPUs) are used. AWS provides pre-built Deep Learning Containers for PyTorch on CUDA, ensuring compatibility. In deployment, an Elastic Kubernetes Service (EKS) or ECS can schedule pods on GPU nodes. During training, tools like PyTorch Lightning could orchestrate multi-GPU training. The system also supports CPU fallback if GPUs are unavailable, although with slower performance.

CI/CD Workflow: We adopt GitHub Actions for continuous integration and delivery. A **CI pipeline** triggers on every pull request: it checks out the code, sets up Python 3.x,

installs dependencies (via requirements.txt), and runs the test suite (pytest). This ensures code quality and catches regressions. A separate **CD pipeline** triggers on merges to main. It builds Docker images for each service, runs any smoke tests, and pushes images to AWS ECR or Docker Hub. It then deploys the updated containers to the cloud infrastructure (for example, by SSH-ing into the EC2 host and running docker-compose up --build -d, or by updating a Kubernetes deployment with the new image tags). Secrets (AWS credentials, DB passwords) are stored securely in GitHub. This automated workflow enforces repeatable, zero-downtime deployments of the microservices stack. Monitoring and alerting (using Prometheus/Grafana or AWS CloudWatch) are added to track system health and model performance over time.

Risk Assessment

Technical Risks

- **Adversarial Attacks:** Like all deep learning systems, the detector is vulnerable to adversarial manipulations of inputs. Small perturbations to a frame or audio waveform could force misclassification. Such attacks undermine the detector's reliability by hiding fake artifacts, so continuous adversarial training and robustness testing are required.
- **Generalization and Concept Drift:** Deepfakes evolve rapidly (new generative models appear frequently), so a model trained on existing datasets may fail on novel methods or unforeseen artifact types. As noted in surveys, models that generalize well in controlled tests can fail on "in-the-wild" fakes. This risk demands modular re-training and periodic updating of models with fresh data.
- **False Positives/Negatives:** No detector is perfect. High false positives (labeling real content as fake) could erode user trust or lead to censorship disputes. False negatives (missing a fake) could give users a false sense of security. Rigorous threshold tuning, ensemble methods, and human-in-the-loop review can mitigate, but some error rate must be accepted as a technical limitation.
- **Performance and Scalability:** Processing video (decoding, CNN inference, etc.) is resource-intensive. Long videos or many simultaneous requests could overwhelm GPU resources or throughput. The system must implement queueing (rate-limiting) and horizontally scale (spin up more inference pods) to handle peak loads. Latency could be high for long videos, requiring user notifications or batching strategies.
- **Data Security and Integrity:** Input videos may be large or contain malicious payloads (e.g. exploit attempts via crafted files). The video/audio services must sanitize inputs (e.g. scanning for known vulnerabilities) and run in a locked-down container. There is also risk of data leaks: user-provided media are sensitive.

Secure transmission (HTTPS), encryption at rest, and strict access controls are mandatory. Model IP theft is another concern (protect weights).

- **Dependency and Integration Failures:** Each microservice and ML model has dependencies. Version mismatches (e.g. PyTorch/CUDA) can break containers. CI/CD helps catch this, but a broken component in production (e.g. metadata parser fails on an unexpected format) could crash the pipeline. Robust error handling and fallbacks (e.g. skip metadata if parser fails) are needed.
- **Explainability Limitations:** Grad-CAM and other XAI methods provide rough visual cues, but they can be misleading. Users might over-interpret highlighted regions. Ensuring the explainability module is tested and calibrated is important, but it cannot guarantee fully understandable rationales for every decision.

Legal Risks

- **Copyright and IP:** Our system processes potentially copyrighted audio/video. Under U.S. law and international treaties, there are “fair use” exceptions for analysis, but this depends on jurisdiction. The Northwestern JTIP blog notes that existing copyright law “falls short in addressing” AI-generated content. If the platform stores or hosts copyrighted media without proper licensing, it could face infringement claims. Solutions include using ephemeral processing (do not store user files longer than necessary) and obtaining legal counsel on fair-use boundaries.
- **Privacy and Personal Data:** Uploaded media often contain personal data (faces, voices). GDPR/CCPA and similar laws treat biometric and media data as sensitive. We must obtain user consent, minimize data retention, and possibly offer data deletion. EXIF data can contain GPS and device identifiers; extracting and storing these carries privacy risks. If the metadata analyzer exports or logs such data, it should be done in compliance with privacy law.
- **Right of Publicity:** Deepfakes exploit individuals’ likenesses. The law is evolving: for example, a proposed U.S. bill (“No AI Fraud Act”) would make it illegal to use a person’s likeness or voice in AI-generated content without consent. Since our educational sandbox allows creation of deepfakes, we must ensure it does not aid illegal impersonation. This may require age-gates and explicit disclaimers, and possibly restricting uploads of celebrity faces.
- **Defamation and Misrepresentation:** If our detector incorrectly labels a real person’s video as fake (or vice versa), it could cause reputational harm. Current defamation law struggles with AI content, so liability is unclear. We must include legal disclaimers (“for research/educational use only” and “not a certified forensic tool”) and require users to agree not to rely on detections as absolute

truth. Logging each decision (with timestamps and model version) helps in an audit trail if disputes arise.

- **Data Scraping and Compliance:** If the system scrapes social media metadata, it must obey site terms of service and API usage policies. Unauthorized scraping could violate laws like the Computer Fraud and Abuse Act. A safer design is to require the user to manually paste any link/metadata they want checked, rather than automated crawling. All network requests should be secure and respect robots.txt or API rate limits.
- **Export Controls and Regulations:** Some jurisdictions (e.g. the upcoming EU AI Act) classify high-risk AI systems like deepfake detectors or generators. We need to monitor regulatory developments; compliance may demand impact assessments or certifications. Using encryption and secure logs aids compliance with audits.

Ethical Risks

- **Non-Consensual Deepfakes:** A major ethical concern is misuse. Deepfakes are often used for harassment, especially “revenge porn” targeting women. Our sandbox demo, while educational, could be abused to create non-consensual content. We must implement safeguards: for example, require that sandbox face inputs be the user’s own face (verified via optical check or manual review), and filter out sexual content. Educational framing and warning screens (as CivAI does) can mitigate harm.
- **Erosion of Trust:** By publicizing how easy deepfakes are to make, the system risks further eroding trust in media. We must emphasize that this tool is for detection/education, not for sensationalizing fakes. Over-reliance on AI scores is also risky – for example, law enforcement using our tool without human oversight might make grave errors. Therefore, output will include confidence levels and encourage human review.
- **Bias and Fairness:** Detection models may perform unevenly across demographics if training data are skewed. For instance, if faces of a certain ethnicity were underrepresented in training, the model might misdetect at higher rates on those faces. This can be perceived as discrimination or censoring. We should audit the system on diverse datasets and disclose known biases. If bias is found, retraining or data augmentation is needed.
- **Dual-Use and Misuse:** Publishing a deepfake sandbox or open APIs can educate, but also inform bad actors. To address this, we restrict access (e.g. tiered subscriptions) and monitor usage patterns. The system should not reveal internal model details (e.g. no decompiled models or gradient API). We should

also follow responsible disclosure: if a user uncovers a new vulnerability or exploit (such as a fooling pattern), have a channel to report it rather than publish it.

- **Psychological Harm:** Viewing one's own face as a deepfake can be unsettling. The UX must provide support (information and opt-outs). Additionally, allowing children to create deepfakes raises safeguarding issues. We must comply with COPPA/children's privacy laws by age-checks and parental consent for minors.
- **Accountability and Transparency:** Users deserve to know how the decisions are made. Black-box results ("fake"/"real") without explanation can be ethically troubling. Hence, the explainability module is crucial. We also log all actions with user consent so any harm can be traced. As Amped notes, forensic evidence needs a clear chain of custody; our logs serve that purpose.

In summary, while the system can provide powerful insight into synthetic media, it must be developed with robust security, strict privacy controls, and clear ethical guidelines. Risk mitigation includes thorough testing (both in-house and third-party audits), clear user agreements, and ongoing monitoring of legal and technical developments.